

Fine-Grained Visual Classification on CUB-200-2011

Romina Raoofian
University of Bonn
Bonn, Germany

s28rrao@uni-bonn.de

Shive Sinaei
University of Bonn
Bonn, Germany

s23ssina@uni-bonn.de

Abstract

We study the effect of four architectural components - pooling strategy, activation function, normalisation, and convolution type - on a LeNet-style network for fine-grained bird species classification using a 25-class subset of CUB-200-2011. Through controlled experiments, we show that normalisation layers yield the greatest accuracy improvement, while depthwise separable convolutions reduce parameters by $1.3\times$ (from 365k to 284k) with only a 1.5% drop in validation accuracy. These findings guide the design of YourNet, a lightweight model within a 500k parameter budget that achieves competitive accuracy on this benchmark.

1. Introduction

Bird species classification is a fine-grained visual recognition problem where the model must distinguish between visually similar species that differ only in subtle patterns.

We work with a 25-class subset of the CUB-200-2011 dataset, which contains high-resolution photographs of bird species. All images are resized to 128×128 pixels and normalised. The provided training split (80% of the full data) is further divided into 80% training and 20% validation subsets, seeded split (seed = 66), ensuring reproducibility.

Our baseline is a LeNet-style CNN consisting of three convolutional blocks, each with a Conv2d, Batch Normalisation, ReLU, and MaxPool layer, followed by three fully connected layers mapping to 25 output classes. The baseline has 365,849 parameters and is trained with the Adam optimiser ($\text{lr} = 10^{-3}$) for up to 50 epochs with batch size 64, achieving a best validation accuracy of 33.08% at epoch 24.

Starting from this baseline, we systematically study four architectural modifications:

- A comparison of max pooling, average pooling, and strided convolutions.
- An evaluation of ReLU, Leaky ReLU, and GELU activations.
- A comparison of Batch Normalisation and Group Normalisation against a no-norm baseline.

- A parameter-efficient variant using depthwise separable convolutions.
- *YourNet*, a final architecture within the 500k parameter budget.

2. Experiments and Discussion

2.1. Task 1: Pooling Strategies

We compare three downsampling strategies while keeping all other components fixed. **AvgPool** replaces each MaxPool with average pooling (kernel 2). **Strided Conv** sets stride = 2 on each Conv2d and removes the pooling call entirely. Parameter count is identical across all variants (365,849).

Variant	Val Acc. (%)	Epoch	Time (s)
Baseline (MaxPool)	33.08	24	158.1
AvgPool	33.85	49	159.6
Strided Conv	33.08	15	142.8

Table 1. Task 1 — Pooling strategies.

All three variants achieve nearly identical accuracy. MaxPool and strided convolutions reach the same peak (33.08%), while AvgPool gains a marginal 0.77% improvement. This confirms that on a small, hard dataset the pooling choice is not a bottleneck. Notably, strided convolutions converge faster (epoch 15 vs. 24) and train slightly quicker (142.8s) since the learned stride replaces a separate pooling operation. AvgPool takes the most epochs (49) to reach its best, suggesting slightly slower convergence.

2.2. Task 2: Activation Functions

All ReLU activations (in conv blocks and FC layers) are replaced with the target function. **Leaky ReLU** uses negative slope $\alpha = 0.1$. **GELU** with its probability according to a Gaussian normal distribution.

This is the most impactful single change across all tasks. Leaky ReLU improves accuracy by +5.38% over the baseline, the largest gain observed in any experiment. ReLU

Variant	Val Acc. (%)	Epoch	Time (s)
Baseline (ReLU)	33.08	24	158.1
Leaky ReLU	38.46	21	158.0
GELU	35.38	10	156.9

Table 2. Task 2 — Activation functions. All variants: 365,849 parameters.

suffers from dying neurons: units whose pre-activations are always negative, contribute zero gradient and stop learning. Leaky ReLU prevents this with a small slope ($\alpha = 0.1$) for negative inputs, which is particularly beneficial on a fine-grained task where gradient signal is weak. GELU also improves over ReLU (+2.3%) and converges fastest (epoch 10), but its peak accuracy is lower than Leaky ReLU, suggesting it finds a good solution quickly but a less optimal one.

2.3. Task 3: Normalisation Layers

Three configurations are evaluated: a network with no normalisation at all as our baseline, one with Batch Normalisation, and one with Group Normalisation. For GroupNorm, we use $G = 8$ groups. While 32 is the largest value that divides evenly into all three channel counts (32, 64, 128), it would reduce each group to a single channel in the first block, losing the benefit of group statistics. We choose $G = 8$ as it gives groups of 4, 8, and 16 channels respectively which is large enough for stable normalisation while keeping the groups meaningful.

Variant	Val Acc. (%)	Epoch	Params
No Norm	31.54	15	365,401
BatchNorm	33.08	24	365,849
GroupNorm (G=8)	35.38	20	365,849

Table 3. Task 3 — Normalisation layers.

Removing normalisation entirely drops accuracy to 31.54% and converges early (epoch 15), indicating unstable training. BatchNorm restores the baseline performance (33.08%), confirming that normalisation is necessary for stable gradient flow. Surprisingly, GroupNorm outperforms BatchNorm by +2.3%, reaching 35.38%. This is likely because at batch size 64 with only a few hundred images per class, the per-batch statistics of BatchNorm can be noisy, while GroupNorm’s channel-group statistics are independent of batch composition and thus more consistent across updates.

2.4. Task 4: Depthwise Separable Convolutions

Every Conv2d block is replaced with a depthwise separable block: depthwise Conv2d (groups = C_{in} , $k=3$, pad=1) $\rightarrow$

BN $\rightarrow$ ReLU $\rightarrow$ pointwise Conv2d (1×1) $\rightarrow$ BN $\rightarrow$ ReLU. This factorises the spatial and channel mixing steps, reducing parameters.

Variant	Val Acc. (%)	Epoch	Params
Baseline	33.08	24	365,849
DWS Conv	31.54	30	284,026

Table 4. Task 4 — Depthwise separable convolutions.

The DWS model reduces parameters by $1.3\times$ (365,849 $\rightarrow$ 284,026) with an accuracy drop of only 1.54%. The slight accuracy drop is expected: the factorised convolution is less expressive than a full Conv2d, but the trade-off is favourable for resource-constrained settings. Convergence is slower (epoch 30 vs. 24), suggesting the model needs more iterations to compensate for reduced capacity.

2.5. Task 5: YourNet

2.5.1. Data Augmentation

For this task, we applied a custom augmentation pipeline.

Since the dataset contains images with varying aspect ratios, directly resizing all images to a fixed resolution could introduce distortions and potentially remove fine-grained features that are important for bird classification. To address this issue, we first padded each image to a size of 500×500 pixels before applying a random resized crop. This preserves the original aspect ratio while still allowing the network to observe images at different scales.

The random cropping operation also improves robustness to variations in bird location, as the birds are not perfectly centered in all images. In addition, we applied horizontal flipping and random rotations to make the model more robust to changes in viewpoint and camera orientation. Finally, colorjitter was used to introduce variations in brightness, contrast, saturation, and hue. This helps the network become less sensitive to illumination changes, preventing it from relying on lighting conditions rather than meaningful bird characteristics.

2.5.2. Model

We modified the baseline LeNet architecture and named the resulting model *YourNet*.

First, we replaced the ReLU activation function used in the baseline with LeakyReLU, as suggested by the results obtained in Task 1.

Furthermore, we introduced dropout layers in the classifier head. Since the CUB-25 dataset is relatively small, dropout acts as a regularization technique that helps reduce overfitting and improves generalization performance.

To increase the representational power of the network, we extended the architecture from three convolutional layers to five convolutional layers. The additional layers allow

the model to learn more complex and hierarchical visual features.

Finally, to control the overall number of parameters despite the increased depth, we reduced the output size of the adaptive average pooling layer from 4×4 to 2×2 . This significantly reduces the dimensionality of the feature vector passed to the fully connected layers while preserving high-level semantic information.

2.5.3. Results

The proposed modifications improved classification performance from 33% for the baseline model to 43% for the proposed YourNet architecture.

To isolate the effect of data augmentation, we also trained the baseline LeNet model using the same augmentation pipeline. In this setting, the baseline model achieved 35.38% accuracy, showing that data augmentation alone provides a noticeable improvement over the original baseline.

However, the full YourNet architecture with a total of 490,837 parameters achieved the best performance, indicating that both the augmentation strategy and architectural improvements contributed to the final results.

3. Conclusion

Overall, the experiments show that activation functions and normalization layers have the greatest impact on performance, while pooling strategies contribute only marginal improvements. Replacing ReLU with Leaky ReLU produced the largest single gain, increasing validation accuracy from 33.08% to 38.46%, while Group Normalization further improved performance compared to Batch Normalization. Depthwise separable convolutions demonstrated that parameter efficiency can be improved with only a small loss in accuracy. Combining the most effective architectural choices with a carefully designed data augmentation pipeline led to the proposed *YourNet*, which achieved the best overall result of 43.00% validation accuracy, representing an improvement of nearly 10 percentage points over the baseline model.